

A. 圆 (circle)

枚举四个点的时间复杂度是 $O(n^4)$ 的，我们希望快速判定一组 A, B, C, D 是否合法。

判定一个圆是否在一个矩形内只需要判断圆的上端、下端、左端、右端在矩形内。

判定一个圆 O_1 是否包含另一个圆 O_2 ，需要：

- 圆 O_1 包含圆 O_2 的圆心。
- 圆 O_1 上距离圆 O_2 的圆心最近的点的距离 $>$ 圆 O_2 的半径。

所有的限制都容易使用纯整数运算判断，使得其无浮点数误差。

时间复杂度： $O(n^4)$ 。

B. 序列 (sequence)

首先序列中一定有元素 n ，因为最后一个放置的积木一定在顶端。

考虑元素 $n - 1$ ，其也必须在序列中存在，且与 n 相邻。

考虑元素 $n - 2$ ，他会被放置在 $n - 1$ 相邻的一个位置上。如果他被放置在 n 的位置上，则不在序列中出现；否则会出现。

经过上述分析，我们发现：从大到小考虑所有数，一个数如果在序列中出现，那一定为目前的这些已出现元素的开头或结尾。

于是我们设计状态： $dp_{i,j}$ 表示从大到小考虑了 i 个数，第 i 个数（颜色为 $n - i + 1$ 的）需要在最终序列中出现，一共有 j 个考虑的数在最终序列中出现，这 j 个数的选择方案与相对排列的组合的数量。

那么答案就是 $\sum_{i=2}^n \sum_{j=2}^k dp_{i,j}(k - j + 1)$ 。

对于第 i 个数与上一个在序列中出现的数相邻，转移为 $dp_{i-1-2k,j-1} \rightarrow dp_{i,j}, k \geq 0$ 。

对于第 i 个数与上一个在序列中出现的数不相邻，转移为 $dp_{i-j+1-2k,j-1} \rightarrow dp_{i,j}, k \geq 0$ 。

暴力实现上述转移的时间复杂度为 $O(n^3)$ ，但可以使用前缀和优化，做到时间复杂度 $O(n^2)$ 。

C. 多边形 (polygon)

首先容易画图得出，最少的操作次数为两端均不为 x 的对角线数量。

考虑一条对角线什么时候可以直接变为一端为 x 的对角线。

令这条对角线连接了 p, q 两点。若存在对角线 $(p, x), (q, x)$ 时，就可以将该对角线变为一端为 x 。

因为如果有 $(p, x), (q, x), (p, q)$ ，考虑 (p, q) 的不经过 x 的弧中，其所在的三角形的另一端点 r 。

那么就有 $(p, x), (q, x), (p, q), (p, r), (q, r)$ ，可以删掉 (p, q) 加入 (r, x) 。

将所有对角线写成不经过 x 的弧的形式，那么：

- 任意两个弧要么包含要么交的长度 $= 0$ 。
- 每次操作都是选择一个不被别的弧包含的弧（极大弧），并删去。

那么我们建立一棵森林，满足包含 p 的弧的点恰好为 p 的所有祖先，方案数即为该森林的拓扑序数量。

一棵 m 个点的森林的拓扑序数量为 $\frac{m!}{\prod s_{z_i}}$ 。这里 $m = ans$ ，即两端均不为 x 的对角线数量。

对于一个点 p , sz_p 就是 p 对应弧所包含的弧的数量 (包括其本身), 那么就是 p 对应弧的弧长 -1 。

于是只需要计算所有对角线不经过 x 的弧的弧长 -1 的乘积。

维护每个点作为 x 时的上述乘积, 那么插入一个对角线 (p, q) , 进行的操作就是两个环上区间乘。段环为链后就是三个链上区间乘。删除对角线就是乘逆元。可以使用树状数组或线段树做到时间复杂度 $O((n+q)\log n)$ 。

D. 道路维修 (road)

初步分析

我们将原图的所有联通块用一个区间 $[l, r]$ 表示, 其含义为这个联通块在南北向上覆盖的区间。那么将第 x 条东西向道路打通的操作, 其实质上就是使所有区间包含 x 的联通块联通。

把每个点对应到各自的连通块, 并求出每个连通块对应到的区间, 那么问题就变为:

- 有 $O(nm)$ 个区间。
- 你选择一些点 x , 令两个区间有边当且仅当他们都包含一个选了的点 x 。(选择一个点的代价为 1 或 2)。
- 询问为: 给定若干个区间, 需要选择一些点 x 使给定的区间互相联通, 最小化选的点的权值和。
(注意未给定的区间也参与连边)

首先缩减询问所给定的区间集合: 仅需保留所有极小的区间。因为如果一个给定的区间包含了一个更小的给定的区间, 那么当更小的区间符合要求时, 大区间的要求就天然满足了, 因此不需要管该大区间的限制, 可以从给定的区间集合中将其删去。

缩减完后, 询问给定的区间没有包含关系, 可以从左到右排序, 满足 " l 单调上升, r 单调上升"。

需要特判掉缩减完后只剩一个区间的情况, 此时答案为该区间内权值最小的点的权值。

算法一

考虑只有两个区间且所有点权值为 1 的问题。

从一个靠左的区间走到一个靠右的区间, 每次在 "当前区间联通块右端点的最大值" 处选点, 直到该最大值 $\geq$ 靠右区间的右端点, 此时只需要再选一个点即可。

只要知道最后一个选择的点是什么, 就可以知道目前的区间联通块右端点的最大值, 即可知道下一个选择的点是什么。

于是设 $f_{i,j}$ 表示目前最后一个选的点在位置 i , 之后选 2^j 个点, 最后一个选的点是什么。

求答案只要求从 r_1 开始至少跳几步能 $\geq r_2$, 容易使用倍增数组做到单 $\log$ 。

时间复杂度: $O(\sum T_k \log n + n \log n + nm)$, 期望通过子任务 1, 2, 4, 获得 35 分。

算法二

考虑所有点权值为 1 的问题。

每个区间都得有一个被选择的点, 并且最靠左的区间得与最靠右的区间联通。

仍然类似的, 从最靠左的区间的右端点出发, 每次尽可能选择最靠右的点。不过如果马上要选的点与上一个选了的点之间夹着一个完全没有点的线段, 就爆了。此时下一个取的点需要是所有区间内没有点的区间的最小右端点。

在实际维护的时候，我们依次考虑从第 i 个区间走到第 $i + 1$ 个区间。在此过程中不断往最右的地方跳，直到再往后跳就要比 r_{i+1} 大了。此时如果第 $i + 1$ 个区间被覆盖了，那就正常往右跳，否则就需要跳到 r_{i+1} 。

时间复杂度： $O(\sum T_k \log n + n \log n + nm)$ ，期望通过子任务 1, 2, 3, 4, 5，获得 65 分。

算法三

现在有的点权为 1，有的点权为 2，那我们把点权为 2 当做选择两个点，然后试图沿用原来的倍增。

但问题是要选 2^k 个点不一定能拆成先选 2^{k-1} 个点再选 2^{k-1} 个点，而可能出现先选 $2^{k-1} - 1$ 个点再选 $2^{k-1} + 1$ 个点的形式。

于是我们魔改倍增数组，令 $dp_{i,j,0/1}$ 表示目前最后一个选的点是 i ，之后选 $2^j - 0/1$ 个点，最后一个选的点是什么。

这样设置的倍增数组就能够转移了。

我们在询问的时候也不再只维护“跳 ans 步能跳到的最靠右点”，而是维护“跳 $ans - 0/1$ 步能跳到的最靠右点”。可以使用倍增数组以及“跳 $1/2$ 步到达的点”来更新该信息。

时间复杂度： $O(\sum T_k \log n + n \log n + nm)$ ，期望通过所有子任务，获得 100 分。